

# Agent SDK headless를 “판단자(Judge) 루프”로

현재 룰 엔진은 문자열 술어(commandMatches, pattern)로만 평가한다. 이것이 잡지 못하는 것 — 의미적 trigger, 우회 동작, 누락된 단계, 트레이스 전반에 대한 NL 질의 — 을 **Haiku 4.5 headless** 호출로 보완한다. 정적 룰과 LLM judge는 두 계층으로 공존하며, prompt caching으로 운영 비용을 100배 가까이 압축한다.

대상 모듈 packages/server/src/governance/    새 하위 모듈 governance/judge/

주력 모델 claude-haiku-4-5-20251001    SDK @anthropic-ai/claude-agent-sdk

## 0 TL;DR

새 판단자

**4**

Verdict · Plan · Discoverer · TraceQ&A

예상 턴당 비용

**~\$0.0002**

cache hit 90% 가정, Haiku

새 SQLITE 테이블

**2**

judge\_verdicts, judge\_cache\_keys

MVP까지

**~5d**

Phase 1+2+3만으로 첫 데모 가능

## 1 전체 아키텍처

기존 룰 엔진의 “turn close” 시점을 트리거로 사용한다. 정적 매처가 먼저 돌고, 결과에 관계없이 Judge 큐에 비동기로 작업을 넣어 LLM 평가를 덧붙인다. 두 결과는 같은 rule\_enforcements의 형제 레코드 judge\_verdicts로 저장되어 UI에서 비교 가능하다.

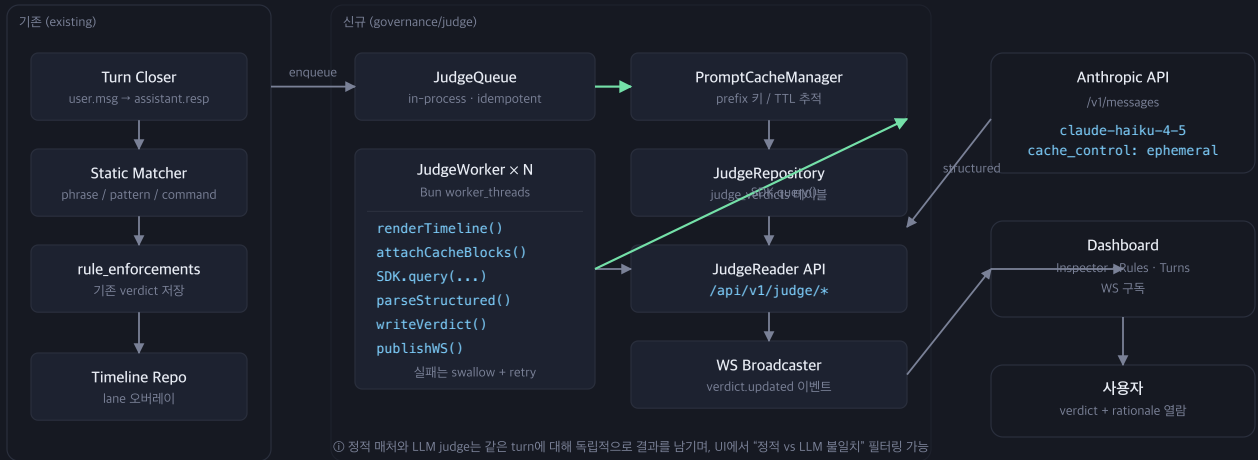


그림 1. Judge loop는 기존 룰 엔진의 turn close에 비동기 piggyback 한다. 메인 경로(에이전트 동작)에 영향 없음.

## 2 네 종류의 Judge

하나의 JudgeWorker가 system prompt와 출력 스키마만 바뀌어서 네 가지 작업을 수행한다. 공통: 같은 **timeline** 렌더링 함수, 같은 **prompt cache prefix**, 같은 구조화 출력 검증(Zod).



그림 2. 4종 Judge는 system prompt + output schema만 다른 한 워커의 변주.

## 3 구체적 유즈케이스 — 실패하는 정적 매칭, 성공하는 LLM

아래 케이스는 모두 “현 정적 매처는 못 잡거나 잘못 잡는데, LLM judge면 깔끔히 풀린다”에 해당.

#### UC-1 의미적 trigger — 룰 단어가 안 박혔는데도 활성화돼야 하는 경우

사용자가 "단위 테스트 좀 보강해줘"라고 했다. 룰의 `trigger.phrases`에는 "add tests", "add unit test"만 등록되어 있다. 정적 매치는 trigger 매칭 실패 → 룰 비활성 → 검증 누락.

정적 매치

trigger phrase 미일치 → **rule 비활성**.  
아무 verdict도 안 남김.

VERDICTJUDGE

의미적 일치 판단 → rule 활성화로 확장.  
turn 종료 시 expectation 평가.

결과: **fail** · evidence: event\_71b...(Write src/auth.ts, no test file written) · rationale: "사용자가 테스트 보강을 요청했으나 \*.test.ts 변경 없음."

#### UC-2 우회 동작 — pattern은 통과하는데 실질적으로 위반

룰: pattern: "\\\\.env\$" 인 파일을 Edit 하지 말 것. 에이전트가 `cat .env > .env.bak`으로 다른 경로에 그대로 복사한다. pattern 매치는 `.env`를 "읽기만" 했다고 봄 → 통과.

정적 매치

`Edit`가 `.env`에 안 닿음 → **pass**.  
실제 위반을 못 잡음.

VERDICTJUDGE (LLM)

timeline에서 ".env 내용이 다른 경로로 복사됨"을 의도 단위로 해석 → **fail**.

결과: **fail** · evidence: event\_a4c...(Bash: cp .env .env.bak) · rationale: "룰 의도는 .env 콘텐츠의 비밀 유지. 단순 경로 매칭 우회."

#### UC-3 Plan vs. Execution — 계획에는 있는데 실제로 안 한 단계

에이전트가 ExitPlanMode에서 4단계 plan을 제시했다: ① 스키마 마이그레이션, ② Service 메서드 추가, ③ 테스트 추가, ④ docs 업데이트. 실행을 보면 ①, ②, ④만 수행. 사용자는 plan을 일일이 비교해야 알 수 있다.

입력

plan(말풍선 4개) + 실제 tool sequence(Edit, Edit, Edit) + Stop.

PLANADHERENCEJUDGE

```
{ score: 0.66, missing: ["3. add tests"], deviated: [] }
```

결과: **0.66** · 누락: "테스트 추가" · UI에 "plan adherence 66%" 메달과 함께 missing 단계 강조.

#### UC-4 암묵적 룰 발굴 — 사용자가 "뭘 검사해야 할지 모르겠는" 상태 해결

초기 사용자의 가장 큰 마찰: 룰 페이지가 비어 있다. 무엇이 "기대 동작"인지 명시적으로 알지 못함. 이미 모인 20개 task에는 풍부한 정답 신호가 있다.

RULEDISCOVERER 입력

최근 20 task의 (prompt 요약 + tool sequence + 변경 파일 카테고리)

RULEDISCOVERER 출력

5개 후보 룰 — UI 큐로 적재, 사람이 "채택/보류/거절".

예시 후보: **"fix 커밋 전 typecheck 누락 7/20", "packages/server 수정 후 vitest 누락 5/20", "docs/\* 단독 변경 시 Read 다수 — 검증 룰 불필요".**

#### UC-5 트레이스 Q&A — "어떻게 동작했는지"를 자연어로 묻기

사용자의 원 목적: "에이전트가 어떻게 동작하는지 확인." 타임라인을 일일이 스크롤하는 대신 대화형으로 묻는다.

질의 예시

- "같은 파일을 두 번 이상 읽은 경우가 있어?"
- "grep 패턴이 의미 있게 좁혀졌나?"
- "계획에 없던 파일 수정은?"
- "이 turn에서 룰을 우회한 정황이 있어?"

응답 (요약)

packages/server/src/work/task/repository/task.repository.ts를 **3회** 재독(11:42, 11:48, 11:53). 첫 grep "task"는 너무 광범위 → 두 번째 "completeTask"로 좁힘. citation: event\_...

UI: Inspector 하단에 chat 입력. 답변의 모든 citation은 timeline event를 클릭 가능한 칩으로 표시.

## 4 시퀀스 — VerdictJudge 1회 실행

turn close에서 dashboard 갱신까지의 시간축. 일반 케이스 800ms 이내, cache miss 케이스 2~3s.

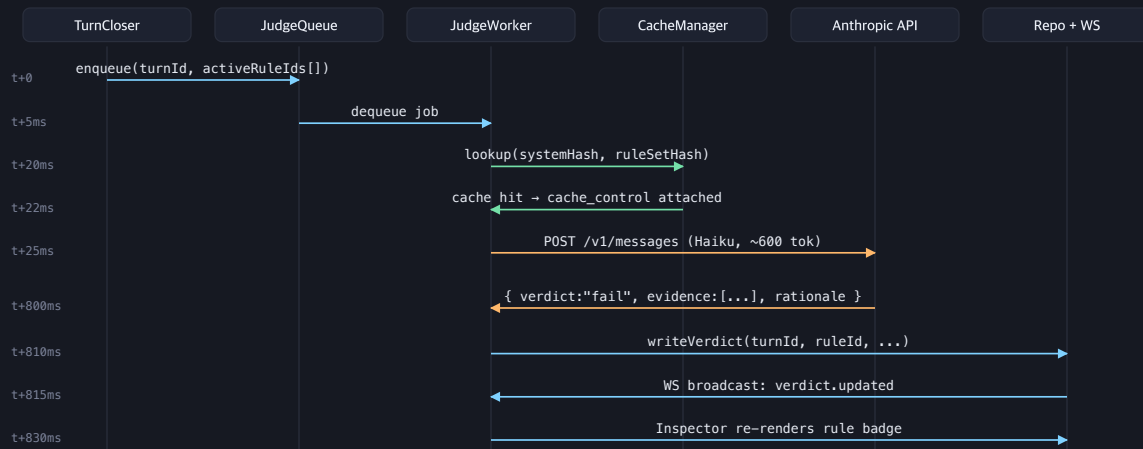


그림 3. cache hit 케이스 — 평균 800ms. 메인 에이전트 경로와 분리되어 있으므로 사용자 체감 지연은 0.

## 5 프롬프트 캐시 전략 — 운영비를 100배 압축하는 곳

Anthropic 캐시는 prefix 매칭. 변하지 않는 가장 큰 **prefix**를 가장 앞에 배치하는 것이 핵심. 그 다음 “자주 같은 값”을 캐시 블록으로 둘러싸고, 마지막에 “이 호출에만 다른 값”을 둔다.

한 요청의 메시지 배치 (왼쪽이 prefix)

① 시스템 프롬프트

JUDGE\_SYS\_PROMPT (불변)

cache\_control: ephemeral · hit 99%

② 룰 카탈로그

활성 룰 N개 JSON (시간당 거의 불변)

cache\_control: ephemeral · hit 95%

③ 타임라인 prefix

같은 task의 이전 turns (재호출 시 누적)

cache\_control: ephemeral · hit 60~80%

④ Δ

이번 turn

+ 질의

호출 1

호출 2

호출 3

↑ 호출이 누적될수록 캐시된 prefix가 길어진다.

그림 4. 4단 캐시 레이아웃. ④만 매번 새로 보내고, ①②③은 같은 task 안에서 거의 항상 hit.

| 입력 구성                 | 토큰 (대략) | CACHE HIT 가정 | 실비용 / 호출  |
|-----------------------|---------|--------------|-----------|
| ① JUDGE_SYS_PROMPT    | 800     | 99%          | \$0.00001 |
| ② 룰 카탈로그 (활성 룰 평균 5개) | 1,200   | 95%          | \$0.00006 |
| ③ task prefix (누적)    | 2,000   | 70%          | \$0.00018 |

| 입력 구성             | 토큰 (대략) | CACHE HIT 가정 | 실비용 / 호출   |
|-------------------|---------|--------------|------------|
| ㉔ 이번 turn + 질의    | 400     | 0%           | \$0.00040  |
| 출력 (verdict JSON) | 200     | —            | \$0.00100  |
| turn당 평균 (롤 1개)   | ~4,600  | —            | ~\$0.00165 |

① 단가는 Haiku 4.5 기준 추정. 캐시된 입력은 약 1/10. 롤이 3개라면 ㉔ 부분만 곱해진다(병렬 호출 가능, 출력만 늘어남). 하루 1,000 turn × 롤 3개 가정 시 ~\$0.5 / day. 운영 부담은 거의 없음.

## 6 데이터 모델

기존 rule\_enforcements를 손대지 않고, 형제 테이블로 추가. UI는 두 테이블의 union으로 verdict를 표현.

```
-- packages/server/db/migrations/20260520_judge.sql
CREATE TABLE judge_verdicts ( id
TEXT PRIMARY KEY, turn_id TEXT NOT NULL, task_id TEXT NOT NULL, judge_kind TEXT NOT
NULL, -- verdict | plan | discoverer | trace_qa rule_id TEXT, -- verdict 전용 verdict
TEXT NOT NULL, -- pass | fail | inconclusive score REAL, -- plan 전용 (0.0-1.0)
evidence_json TEXT NOT NULL, -- citedEventIds + rationale model_id TEXT NOT NULL,
input_tokens INTEGER, output_tokens INTEGER, cache_hit_tokens INTEGER, latency_ms
INTEGER, created_at INTEGER NOT NULL ); CREATE INDEX idx_jv_turn ON
judge_verdicts(turn_id); CREATE INDEX idx_jv_task_kind ON judge_verdicts(task_id,
judge_kind); CREATE TABLE judge_cache_keys ( key_hash TEXT PRIMARY KEY, block TEXT NOT
NULL, -- system | rules | task_prefix scope_id TEXT, -- task_id for task_prefix
first_seen_at INTEGER NOT NULL, last_used_at INTEGER NOT NULL );
```

## 7 구현 계획 (Phase 단위)

총 12 사람-일. **Phase 1-3** 만으로도 첫 데모 (UC-1, UC-2)를 보일 수 있어 ROI가 가장 높음.

P0

### 스캐폴딩

NestJS 하위 모듈 governance/judge, SQLite 마이그레이션, Zod 출력 스키마, JudgeKind enum.

1 day

### VerdictJudge MVP (룰 1개, 캐시 없이)

P1

turn close 이벤트 hook → 활성 룰 fetch → Haiku 호출 → judge\_verdicts insert → WS broadcast.

2 days

### Prompt cache 적용

P2

4-블록 레이아웃, PromptCacheManager, judge\_cache\_keys 기록, hit-rate 메트릭.

1 day

### UI 통합 — Rules 탭에 “LLM verdict 토글”

P3

같은 turn에 대해 정적 vs LLM 결과를 좌우로 표시. 불일치 필터, rationale 펼치기, citation 클릭.

1 day

### PlanAdherenceJudge

P4

ExitPlanMode + Stop 짝짓기 → plan 텍스트 추출 → executed sequence 직렬화 → score + missing/deviated 산출.

2 days

### RuleDiscoverer (야간 배치)

P5

Project별 cron, Sonnet 모델, 후보 룰 큐 UI(채택/보류/거절), 채택 시 자동 POST /api/v1/rules.

2 days

### TraceQAJudge — Inspector chat

P6

multi-turn 대화 상태 관리, citation 칩, “timeline에 강조 표시” 인터랙션.

3 days

### Backfill — 과거 turns 재평가

P7

POST /api/v1/rules/:id/re-evaluate가 이미 존재. LLM judge도 같은 트리거로 backfill.

1 day · opt

```
// 추가 (기존 /api/v1 라우트 패밀리에 합류)
POST /api/v1/judge/turns/:turnId/run // 명시적 재실행
GET /api/v1/judge/verdicts?taskId=... // 필터: kind, ruleId, verdict
GET /api/v1/judge/verdicts/:id // 단건 (rationale + evidence)
POST /api/v1/judge/discover/:projectId // 룰 발굴 트리거
GET /api/v1/judge/candidates // 후보 룰 큐
POST /api/v1/judge/candidates/:id/accept // 채택 → 실제 룰로 변환
POST /api/v1/judge/ask // TraceQA 단발 / streaming
```

## 9 핵심 코드 스켈레톤

```
// packages/server/src/governance/judge/judge.worker.ts
import { query } from "@anthropic-ai/claude-agent-sdk";
import { z } from "zod";

const VerdictSchema = z.object({
  verdict: z.enum(["pass", "fail", "inconclusive"]),
  evidenceEventIds: z.array(z.string()).max(10),
  rationale: z.string().max(500)
});

export async function runVerdictJudge(job: VerdictJob): Promise<Verdict> {
  const blocks = [
    { type: "text", text: JUDGE_SYS, // ①
      cache_control: { type: "ephemeral" } },
    { type: "text", text: renderRules(job.activeRules), // ②
      cache_control: { type: "ephemeral" } },
    { type: "text", text: renderTaskPrefix(job.taskId), // ③
      cache_control: { type: "ephemeral" } },
    { type: "text", text: renderTurn(job.turn) // ④ Δ
    }
  ];

  const stream = query({
    model: "claude-haiku-4-5-20251001",
    systemPrompt: blocks,
    messages: [{ role: "user", content: renderQuestion(job.rule) }],
    outputSchema: VerdictSchema // SDK가 검증된 JSON 강제
  });

  for await (const ev of stream) {
    if (ev.type === "result") return ev.output;
  }
  throw new Error("judge: no result");
}
```

## 10 리스크와 완화

### LLM 변덕

같은 입력에 다른 verdict. 완화: temperature 0 고정 + 정적 매처를 "1차"로 두고 LLM은 "2차 의견". 두 결과 불일치 시 UI에서 명시.

### Privacy contract 침해

timeline 렌더링이 tool\_result body를 끌어오면 안 됨. 완화: renderTimeline()은 기존 read-only repo만 통해 데이터 접근 — body는 어차피 저장되지 않음.

### 비용 폭주

cache miss가 누적되면 ☹ blob이 매번 신규. 완화: task별 daily token quota, hit-rate <50% 시 알람, Discoverer는 야간 한정.

### 응답 지연

Anthropic API rate limit / 일시 장애. 완화: queue + 지수 backoff, "LLM verdict pending" 상태를 UI에서 명시, 24h 후에도 미도착이면 inconclusive.

### 구조화 출력 위반

SDK outputSchema가 reject할 때. 완화: 1회 재시도 → 그래도 실패면 inconclusive로 기록. metric 추적.

### Discoverer 환각

존재하지 않는 패턴을 룰로 제안. 완화: 사람 채택 필수(자동 활성화 없음) + 후보에는 "근거 task 3개" 인용 강제.